Matrix Compression via Randomized Low Rank and Low Precision Factorization

Rajarshi Saha, Varun Srivastava, Mert Pilanci (Stanford University)

NEURAL INFORMATION PROCESSING SYSTEMS

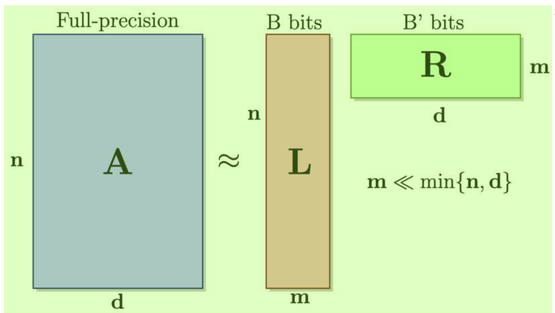
Motivation

- Matrices are ubiquitous -- can involve billions of elements making their storage and processing quite demanding in terms of computational resources and memory usage. Eg.: Vector databases, Kernel matrices, LLM weight matrices, etc.
- Question: How to compress matrices via **dimensionality reduction** and **quantization**?
- Several real-world matrices exhibit **approximately low-rank structure** due to inherent redundancy or patterns.

$$\mathbf{A} = \sum_{i=1}^{\mathrm{rank}(\mathbf{A})} \sigma_i \mathbf{u}_i \mathbf{v}_i^\top$$

Leverage the approximate low-rank structure while quantizing matrices

Our Algorithm: Low-Precision and Low Rank



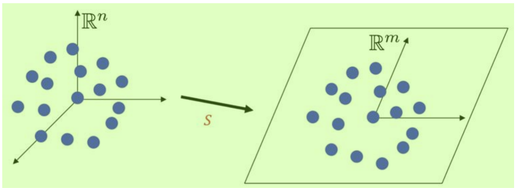
Algorithm 1: **LPLR**: Randomized Low-Precision Low-Rank factorization

Input : Matrix $\mathbf{A} \in \mathbb{R}^{n\times d}$, sketch size m, Quantizers Q, Q′ with dynamic ranges $\mathrm{R_Q}$, $\mathrm{R_{Q'}}$ and bit-budgets B, B′ respectively.
Output : Factorization: $\mathbf{LR}$ where $\mathbf{L} \in \mathbb{R}^{n\times m}$, $\mathbf{R} \in \mathbb{R}^{m\times d}$

1. Sample a Gaussian sketching matrix $\mathbf{S} \in \mathbb{R}^{d\times m}$ with entries $S_{ij} \sim \mathcal{N}\left(0, \frac{1}{m}\right)$.
2. Compute an approximate basis of column space of $\mathbf{A}$ by forming the sketch: $\mathbf{AS}$.
3. Quantize the approximate basis with Q to get $\mathrm{Q}(\mathbf{AS})$.
4. Find $\mathbf{W}^* = \arg\min_{\mathbf{W}} \|\mathrm{Q}(\mathbf{AS})\mathbf{W} - \mathbf{A}\|_{\mathrm{F}}^2 = \mathrm{Q}(\mathbf{AS})^\dagger \mathbf{A}$.
5. Quantize $\mathbf{W}^*$ using quantizer Q′ to get $\mathrm{Q}'(\mathbf{W}^*)$.
6. **return** *Low-rank and low-precision approximation* $\mathbf{LR}$ *where* $\mathbf{L} = \mathrm{Q}(\mathbf{AS})$, $\mathbf{R} = \mathrm{Q}'(\mathbf{W}^*)$.

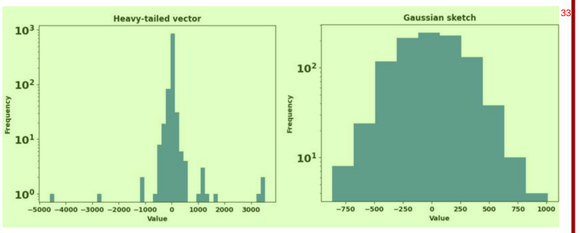
- We factorize, $\mathbf{A} \approx \mathbf{LR}$, where the entries of $\mathbf{L}$ and $\mathbf{R}$ are quantized with $\mathbf{B}$ and $\mathbf{B'}$ bits per entry respectively.
- Compression ratio with respect to naive quantization (uniformly allocate $\mathbf{B_{nq}}$ bits per entry) is $\frac{mn\mathrm{B}+md\mathrm{B}'}{nd\mathrm{B_{nq}}}$.
- By tuning sketch-size we can ensure compression ratio ≤ 1 for $\mathrm{B_{nq}} = 1$, while letting B and B′ to take values allowed by current hardware-primitives, e.g., 4 -- bits, 8 -- bits, etc.
- Low-precision computations **are faster**: Computing $\mathbf{Ax}$ versus $\mathbf{L}(\mathbf{Rx})$.

Randomized Embeddings



$$(1-\epsilon)\|\mathbf{z}\|_2 \le \|\mathbf{Sz}\|_2 \le (1+\epsilon)\|\mathbf{z}\|_2 \quad \forall\, \mathbf{z} \in \mathcal{V}$$

- **Subspace embeddings**: Norm of vectors in the range space is approximately preserved with high probability when $m \gtrsim \epsilon^{-2}\mathrm{rank}(\mathbf{A})$.
- **Critical dimension**, m does not depend on original dimension, n.



- Random Embeddings (e.g., Gaussian) also help in quantization by reducing the dynamic range.

$$\|\mathbf{Sx}\|_\infty \le \mathrm{O}\left(\frac{\|\mathbf{x}\|_2}{\sqrt{d}}\right) \quad \text{or,} \quad \|\mathbf{Sx}\|_\infty \le \mathrm{O}\left(\sqrt{\frac{\log d}{d}}\|\mathbf{x}\|_2\right)$$

Data compression and Nearest Neighbors

Compressing a brain MRI. $\mathrm{B} = 4$, $\mathrm{B}' = 8$, $\mathrm{B_{nq}} = 1, m = 124$, $n = 1534$ $d = 1433$

DSVD: Computes the best rank-k approximation and quantize the factors independently.
LSVD: Variant of LPLR that computes exact SVD instead of randomized SVD.

- For a given data matrix $\mathbf{A} \in \mathbb{R}^{n\times d}$ and a query $\mathbf{x} \in \mathbb{R}^d$, retrieve

$$i^* = \mathrm{argmax}_{i\in[n]}(\mathbf{Ax})_i \;\approx\; \mathrm{argmax}_{i\in[n]}(\mathbf{LRx})_i$$

- Applications: Semantic search over vector databases (music recommendation), In-context learning for LLMs, etc.

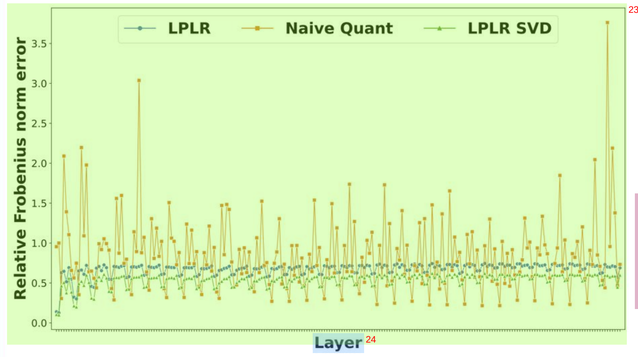
Table 5: **CIFAR100 embeddings generated by MobileNetV3 with an unquantized accuracy and F1 score** 76%:Results on LPLR and LPLR-SVD with $\mathrm{B} = \mathrm{B}' = 8$ bits

	Frobenius Norm Error				Accuracy (%)				Weighted F1 Score (%)			
$\mathrm{B_{nq}}$	**LPLR**	**LSVD**	DSVD	NQ	**LPLR**	**LSVD**	DSVD	NQ	**LPLR**	**LSVD**	DSVD	NQ
1	**1.04**	1.08	1.09	6.75	79	**82**	**82**	1	79	**82**	**82**	0
2	**1.08**	1.1	1.12	2.18	**80**	**80**	**80**	1.7	**80**	**80**	**80**	1.3
4	**1.11**	1.12	1.14	1.17	**79**	78	77	75	**79**	78	78	75

Table 6: **IMDB embeddings generated by BERT with an unquantized accuracy and F1 score** 75% **and** 74% **respectively**: Results on LPLR and LPLR-SVD with $\mathrm{B} = \mathrm{B}' = 8$ bits

	Frobenius Norm Error				Accuracy (%)				Weighted F1 Score (%)			
$\mathrm{B_{nq}}$	**LPLR**	**LSVD**	DSVD	NQ	**LPLR**	**LSVD**	DSVD	NQ	**LPLR**	**LSVD**	DSVD	NQ
1	0.313	**0.241**	0.229	6.63	73	74	**75**	50	74	74	**75**	33
2	0.235	0.178	**0.161**	1.016	**74**	**74**	**74**	50	**74**	**74**	**74**	50
4	0.148	0.122	**0.098**	0.417	**75**	74	**75**	73	74	74	**75**	73

Compression of Large Language Models



$\mathrm{B} = \mathrm{B}' = 8$ bits, $\mathrm{B_{nq}} = 4$ bits			
Metric	**LPLR**	**LPLR-SVD**	Naive Quant.
Mean	0.672	**0.537**	0.836
Std Dev	0.080	**0.079**	0.470

Compressing the weight matrices of LlaMa. We consistently observe better Frobenius norm error, except for specific layers.

Approximation Error Guarantees

Table 1: Comparison with baselines (row-norm bound is constant, i.e., $\|\mathbf{a}^{(i)}\| = \mathrm{O}(1)$). $k, m \ll \min\{d, n\}$. n: no. of rows, d: no. of columns, m: sketch size, ϵ: error tolerance, $\delta = k/(m-k-1)$. The expressions for bit-budget (per entry) ignores constant multiplicative factors inside the $\log_2(\cdot)$. We assume $n \ge d$.

Algorithms	Approximation error	Bit-budget (per entry)	Computation
Naïve uniform	ϵ	$\frac{1}{2}\log_2\left(\frac{nd}{\epsilon}\right)$	$\mathrm{O}(nd)$
Direct-SVD	$\|\mathbf{A}_k - \mathbf{A}\|_{\mathrm{F}}^2 + \epsilon$	$\frac{1}{2}\log_2\left(\frac{k\sigma_1^2}{\epsilon}\sqrt{nd}\right)$	$\mathrm{O}(nd^2)$
LPLR *(ours)*	$(1+\delta)\|\mathbf{A}_k - \mathbf{A}\|_{\mathrm{F}}^2 + \epsilon$	$\frac{1}{2}\log_2\left(\frac{\kappa(\mathbf{A}_k)\kappa}{\epsilon}\frac{nm}{\sqrt{d}}\sqrt{\log\left(\frac{mn^2}{\epsilon}\right)}\right)$	$\mathrm{O}(ndm)$

R. Saha, V. Srivastava, M. Pilanci, "**Matrix Compression via Randomized Low Rank and Low Precision Factorization**", *37th Conference on Neural Information Processing Systems (NeurIPS), 2023.*

Github